

Stats 292 — Statistical Models of Text and Language

Homework: Part-of-Speech Tagging via Hidden Markov Models

Course: Stats 292, Prof. David Donoho, Spring 2026

Based on: Manning & Schütze, *Foundations of Statistical Natural Language Processing*, Ch. 10 (§10.1–10.2); Jurafsky & Martin, *Speech and Language Processing*, 3rd ed., Ch. 8

Data sources: NLTK Brown Corpus (Francis & Kučera 1964); BSKY2GBQ Bluesky firehose archive

About the Data

Brown Corpus (Henry Kučera & W. Nelson Francis, Brown University 1964): One million words of American English sampled from 500 texts across 15 genres. Each word is annotated with a part-of-speech tag. We use the **Universal tagset** (Petrov, Das & McDonald 2012), which collapses the original 87 Brown tags into 12 coarse categories.

Tag	Meaning	Examples
NOUN	Noun	<i>dog, city, happiness</i>
VERB	Verb	<i>run, is, gone</i>
ADJ	Adjective	<i>red, big, happy</i>
ADV	Adverb	<i>quickly, never, well</i>
PRON	Pronoun	<i>I, he, they, who</i>
DET	Determiner	<i>the, a, this, each</i>
ADP	Adposition (preposition)	<i>in, of, on, for</i>
NUM	Number	<i>one, 42, first</i>
CONJ	Conjunction	<i>and, but, or</i>
PRT	Particle	<i>not, up, 's</i>
.	Punctuation	<i>., : ?</i>
X	Other / foreign / unknown	<i>£, etc.</i>

Bluesky skeets: The same 1% sample of English skeets from November 18–24, 2024

(~300,000 posts) used in previous homeworks. Bluesky text is informal and contains hashtags, @-mentions, neologisms, and deliberate non-standard spellings — a challenging test of a tagger trained on edited formal text.

Overview

Part-of-speech (POS) tagging assigns a syntactic category to each word in a sentence. It is the canonical application of Hidden Markov Models in NLP.

The HMM frame (Manning & Schütze §10.1) has three parameter matrices:

$$\pi_t = P(t_1 = t) \quad (\text{initial distribution})$$

$$A_{t' \rightarrow t} = P(t_i = t \mid t_{i-1} = t') \quad (\text{transition matrix})$$

$$B_{t \rightarrow w} = P(w_i = w \mid t_i = t) \quad (\text{emission matrix})$$

Given observed words $\mathbf{w}$, the tagger finds:

$$\hat{\mathbf{t}} = \arg \max_{\mathbf{t}} \pi_{t_1} \prod_{i=1}^n A_{t_{i-1} \rightarrow t_i} \cdot B_{t_i \rightarrow w_i}$$

In this homework you will:

1. Load the Brown Corpus and estimate π , A , B from training data (M&S Figure 10.1).
2. Implement the Viterbi algorithm in **log space** to find $\hat{\mathbf{t}}$ (M&S Figure 10.2).
3. Evaluate on held-out Brown sentences: per-tag accuracy and confusion matrix.
4. Add suffix-based OOV heuristics for words not seen during training.
5. Apply the tagger to Bluesky skeets and analyze failure modes.

Environment Setup

This notebook runs in Jupyter under the `stats292` conda environment. All packages — including `nltk` — are managed via `environment.yml`.

```
conda activate stats292
```

```
jupyter lab
```

The two NLTK corpora below are downloaded once to `~/nltk_data/` and cached for subsequent sessions.

```
In [73]: import nltk
nltk.download('brown', quiet=True)
nltk.download('universal_tagset', quiet=True)

from collections import defaultdict, Counter
import math
import random
```

```
import re
import pandas as pd
```

Part 1 — Training the HMM (Manning & Schütze Figure 10.1)

1a. Load the Brown Corpus

```
In [74]: tagged_sentences = list(nltk.corpus.brown.tagged_sents(tagset='universal'))

print(f"Total sentences: {len(tagged_sentences):,}")
print(f"Total tokens:    {sum(len(s) for s in tagged_sentences):,}")
print(f"\nFirst sentence:")
for word, tag in tagged_sentences[0]:
    print(f" {word:20s} {tag}")
```

Total sentences: 57,340

Total tokens: 1,161,192

First sentence:

The	DET
Fulton	NOUN
County	NOUN
Grand	ADJ
Jury	NOUN
said	VERB
Friday	NOUN
an	DET
investigation	NOUN
of	ADP
Atlanta's	NOUN
recent	ADJ
primary	NOUN
election	NOUN
produced	VERB
`	.
no	DET
evidence	NOUN
'	.
that	ADP
any	DET
irregularities	NOUN
took	VERB
place	NOUN
.	.

1b. Train / Test Split

Hold out the last 20% of sentences for evaluation. **Do not inspect the test set until Part 3.**

```
In [75]: random.seed(42)
```

```

n      = len(tagged_sentences)
split  = int(0.8 * n)
train_sents = tagged_sentences[:split]
test_sents  = tagged_sentences[split:]

print(f"Train: {len(train_sents):,} sentences")
print(f"Test:  {len(test_sents):,} sentences")

```

Train: 45,872 sentences
Test: 11,468 sentences

1c. Count Frequencies

Accumulate the four counts needed to estimate π , A , and B .

```

In [76]: TAGS = ['NOUN', 'VERB', 'ADJ', 'ADV', 'PRON',
                 'DET', 'ADP', 'NUM', 'CONJ', 'PRT', '.', 'X']

tag_counts      = Counter()
start_counts    = Counter()
bigram_counts    = defaultdict(Counter)
emission_counts = defaultdict(Counter)

for sent in train_sents:
    if not sent:
        continue
    words = [w.lower() for w, t in sent]
    tags  = [t          for w, t in sent]

    start_counts[tags[0]] += 1

    for i, (word, tag) in enumerate(zip(words, tags)):
        tag_counts[tag] += 1
        emission_counts[tag][word] += 1
        if i > 0:
            bigram_counts[tags[i-1]][tag] += 1

train_vocab = set(w.lower() for s in train_sents for w, t in s)

print(f"Training vocabulary: {len(train_vocab):,} word types")
print(f"\nTag frequencies:")
for tag, count in sorted(tag_counts.items(), key=lambda x: -x[1]):
    print(f" {tag:6s} {count:7,}")

```

Training vocabulary: 45,755 word types

Tag frequencies:

NOUN	241,528
VERB	150,459
ADP	126,332
.	118,482
DET	116,989
ADJ	73,866
ADV	45,940
PRON	35,550
CONJ	32,177
PRT	23,316
NUM	13,802
X	1,205

1d. Estimate π , A , B with Laplace Smoothing

Add-1 smoothing ensures no probability is zero, preventing Viterbi paths from being killed by a single unseen event.

```

In [77]: num_sentences = len(train_sents)
num_tags = len(TAGS)

# Initial probabilities:  $\pi_t = P(t \text{ at position } 0)$ 
start_p = {
    t: (start_counts[t] + 1) / (num_sentences + num_tags)
    for t in TAGS
}

# Transition probabilities:  $A[\text{prev}][\text{curr}] = P(\text{curr} \mid \text{prev})$ 
trans_p = {}
for prev_tag in TAGS:
    total = sum(bigram_counts[prev_tag].values())
    trans_p[prev_tag] = {
        curr_tag: (bigram_counts[prev_tag][curr_tag] + 1) / (total + num_tags)
        for curr_tag in TAGS
    }

# Emission probabilities: computed on demand (vocabulary too large for full matrix)
vocab_size = len(train_vocab)

def emit_p(tag, word):
    """ $P(\text{word} \mid \text{tag})$  with Laplace smoothing. OOV words get a small floor."""
    word = word.lower()
    count = emission_counts[tag].get(word, 0)
    total = tag_counts[tag]
    if count == 0:
        return 1.0 / (total + vocab_size + 1)
    return (count + 1) / (total + vocab_size)

# Sanity check
print("P(w='the' | tag):")
for tag in TAGS:
    print(f" {tag:6s} {emit_p(tag, 'the'):.6f}")

```

```

P(w='the' | tag):
NOUN    0.000003
VERB    0.000005
ADJ     0.000008
ADV     0.000011
PRON    0.000012
DET     0.375983
ADP     0.000006
NUM     0.000017
CONJ    0.000013
PRT     0.000014
.       0.000006
X       0.000085

```

1e. Inspect the Transition Matrix

The transition matrix A captures English grammar implicitly.

```

In [78]: A_df = pd.DataFrame(
    {curr: {prev: trans_p[prev][curr] for prev in TAGS} for curr in TAGS},

```

```

        index=TAGS, columns=TAGS
    )
    print("Transition matrix A [row = previous tag, col = current tag]")
    print(A_df.round(3).to_string())

```

```

Transition matrix A [row = previous tag, col = current tag]
      NOUN  VERB  ADJ  ADV  PRON  DET  ADP  NUM  CONJ  PRT  .
X
NOUN  0.156  0.156  0.013  0.025  0.019  0.016  0.252  0.009  0.060  0.017  0.277
0.000
VERB  0.102  0.189  0.060  0.104  0.045  0.164  0.175  0.010  0.015  0.062  0.075
0.000
ADJ   0.664  0.018  0.058  0.009  0.003  0.005  0.087  0.007  0.037  0.019  0.092
0.000
ADV   0.034  0.248  0.143  0.099  0.042  0.076  0.145  0.015  0.016  0.028  0.153
0.000
PRON  0.008  0.717  0.010  0.054  0.009  0.017  0.056  0.001  0.011  0.024  0.093
0.000
DET   0.619  0.066  0.245  0.018  0.010  0.006  0.010  0.010  0.001  0.002  0.013
0.001
ADP   0.265  0.043  0.088  0.015  0.058  0.453  0.019  0.032  0.002  0.014  0.010
0.000
NUM   0.373  0.046  0.058  0.020  0.008  0.013  0.131  0.022  0.038  0.005  0.285
0.000
CONJ  0.260  0.182  0.119  0.090  0.055  0.153  0.076  0.020  0.000  0.023  0.021
0.000
PRT   0.038  0.655  0.018  0.028  0.006  0.081  0.085  0.006  0.010  0.009  0.062
0.000
.     0.143  0.124  0.050  0.070  0.066  0.116  0.113  0.023  0.114  0.024  0.154
0.002
X     0.050  0.050  0.004  0.007  0.007  0.006  0.053  0.002  0.021  0.007  0.258
0.537

```

```
In [79]: pd.Series(start_p).nlargest(3)
```

```

Out[79]: DET      0.232456
        NOUN      0.148222
        PRON      0.139177
        dtype: float64

```

Question 1.1. Examine the transition matrix. Which tag most commonly follows DET? Which most commonly follows VERB? Do these match your grammatical intuition about English?

Question 1.2. Which three tags have the highest start probability π ? Does the Brown Corpus genre mix (news, fiction, academic) affect which tags begin sentences?

Question 1.3. Find the transition $A_{t' \rightarrow t}$ with the highest probability. Find the second highest. What grammatical constructions do these encode?

Write your answers here (double-click to edit this cell):

1.1. NOUN most commonly follows DET, VERB most commonly follows VERB. This matches what I'd expect because determiners like the typically point to nouns, and English uses a lot of helping verbs like is which point to other verbs.

1.2. DET, NOUN, PRON have the highest start probability. The genre probably affects the starting distribution a little bit (you can imagine skeets may have less DETs which get dropped in slang), but English conventions with regards to POS likely don't change too much between common unrestrained genres like academic or fiction.

1.3. PRON to VERB has the highest probability at .717. ADJ to NOUN is the second highest at 0.664. PRON-VERB encodes a subject-predicate relation like I am, She is, They ran, etc. ADJ-NOUN is a modifying pair where the adjective is attuning the noun (sweet cake, purple robot, etc).

Part 2 — Viterbi Decoder (Manning & Schütze Figure 10.2)

2a. Log-Space Viterbi

Multiplying many small probabilities causes arithmetic underflow to zero. Work in log space throughout: replace multiplication with addition, $\arg \max$ of products with $\arg \max$ of sums.

```
In [80]: def viterbi(words, tags, start_p, trans_p, emit_fn):
    """
    Viterbi decoding in log space.

    words:    list of observed word strings
    tags:      list of possible POS tags
    start_p:  dict {tag: P(tag at position 0)}
    trans_p:  dict of dicts {prev_tag: {curr_tag: P(curr | prev)}}
    emit_fn:  callable (tag, word) -> P(word | tag)

    Returns:  list of predicted tags, same length as words
    """
    n = len(words)
    if n == 0:
        return []

    # viterbi_scores[t][tag] = best log-prob of any path ending at 'tag' at step t
    # backptr[t][tag]        = which previous tag produced that best path
    viterbi_scores = [{}]
    backptr        = [{}]
```

```
    # Initialise at position 0
    for tag in tags:
        viterbi_scores[0][tag] = (math.log(start_p[tag]) +
                                   math.log(emit_fn(tag, words[0])))
        backptr[0][tag] = None

    # Recurrence
    for t in range(1, n):
```

```

viterbi_scores.append({})
backptr.append({})
word = words[t]

for curr_tag in tags:
    log_emit = math.log(emit_fn(curr_tag, word))
    best_prev, best_score = None, float('-inf')

    for prev_tag in tags:
        score = (viterbi_scores[t-1][prev_tag] +
                 math.log(trans_p[prev_tag][curr_tag]) +
                 log_emit)
        if score > best_score:
            best_score = score
            best_prev = prev_tag

    viterbi_scores[t][curr_tag] = best_score
    backptr[t][curr_tag] = best_prev

# Termination
best_final = max(viterbi_scores[n-1], key=viterbi_scores[n-1].get)

# Backtrace
path = [best_final]
for t in range(n-1, 0, -1):
    path.append(backptr[t][path[-1]])
path.reverse()
return path

```

2b. Smoke Test

```

In [81]: def tag_sentence(words, emit_fn=emit_p):
        return viterbi([w.lower() for w in words], TAGS, start_p, trans_p, emit_fn)

test_sentences = [
    "The dog runs quickly in the park ".split(),
    "Stolen painting found by tree ".split(),
    "Juvenile court to try shooting defendant ".split(),
]

for words in test_sentences:
    predicted = tag_sentence(words)
    print("Words: ", " ".join(f"{w:12s}" for w in words))
    print("Tags:  ", " ".join(f"{t:12s}" for t in predicted))
    print()

```

Words:	The	dog	runs	quickly	in	the
	park	.				
Tags:	DET	NOUN	VERB	ADV	ADP	DET
	NOUN	.				

Words:	Stolen	painting	found	by	tree	.
Tags:	PRON	VERB	VERB	ADP	NOUN	.

Words:	Juvenile	court	to	try	shooting	defendant
	.					
Tags:	ADJ	NOUN	PRT	VERB	VERB	NOUN
	.					

```
In [82]: tag_sentence("Time flies like an arrow".split())
```

```
Out[82]: ['NOUN', 'NOUN', 'ADP', 'DET', 'NOUN']
```

Question 2.1. "Time flies like an arrow" is a classic syntactic ambiguity. What does your tagger produce? What is the alternative parse? Why does the language model prior favor one reading?

Question 2.2. What is the time complexity of the Viterbi recurrence for a sentence of length n with $|T|$ tags? What is the space complexity for storing the backpointer table?

Write your answers here:

2.1. We get [(Time, "NOUN"), (flies, "NOUN"), (like, "ADP"), (an, "DET"), (arrow, "NOUN")]. Other alternatives include [(Time, "NOUN"), (flies, "VERB"), (like, "ADP"), (an, "DET"), (arrow, "NOUN")] or [(Time, "VERB"), (flies, "NOUN"), (like, "ADP"), (an, "DET"), (arrow, "NOUN")]. The model prefers this sequence of tags because it maximizes the joint probability. Even though the Viterbi algorithm explores all the tag combinations via DP, the score is treated as markov which only looks at the previous tag. This means we are locally optimizing at each step so our model prefers high probability bigram transitions and more common word assignments (ie flies being a noun) as long as it leads to high probability tag combinations later on even if the whole sentence isn't grammatical.

2.2. The time complexity is $O(n |T|^2)$ because we loop through all n words then at each word we evaluate all the different current tags $|T|$ and different previous tag $|T|$ which is $|T|^2$ in total. The space complexity is just $O(n|T|)$ because for each word we need to save the score for each of the tags.

Part 3 — Evaluation on Brown Held-Out Set

3a. Per-Token Accuracy

```
In [83]: total_tokens = 0
correct_tokens = 0
gold_tags_all = []
pred_tags_all = []
word_tags_all = []

for sent in test_sents:
    if not sent:
        continue
    words = [w.lower() for w, t in sent]
    gold = [t for w, t in sent]
    pred = tag_sentence(words)

    gold_tags_all.extend(gold)
    pred_tags_all.extend(pred)
    word_tags_all.extend(words)
    correct_tokens += sum(g == p for g, p in zip(gold, pred))
    total_tokens += len(gold)

accuracy = correct_tokens / total_tokens
print(f"Token accuracy on held-out Brown: {correct_tokens:,}/{total_tokens:,} = {ac
```

Token accuracy on held-out Brown: 168,340/181,546 = 0.9273

3b. Per-Tag Precision, Recall, F1

```
In [84]: per_tag = defaultdict(lambda: {'tp': 0, 'fp': 0, 'fn': 0})

for g, p in zip(gold_tags_all, pred_tags_all):
    if g == p:
        per_tag[g]['tp'] += 1
    else:
        per_tag[g]['fn'] += 1
        per_tag[p]['fp'] += 1

print(f"{'Tag':6s} {'Prec':>6} {'Rec':>6} {'F1':>6} {'Support':>8}")
print("-" * 45)
for tag in TAGS:
    tp = per_tag[tag]['tp']
    fp = per_tag[tag]['fp']
    fn = per_tag[tag]['fn']
    prec = tp / (tp + fp) if (tp + fp) > 0 else 0.0
    rec = tp / (tp + fn) if (tp + fn) > 0 else 0.0
    f1 = 2 * prec * rec / (prec + rec) if (prec + rec) > 0 else 0.0
    supp = tp + fn
    print(f"{tag:6s} {prec:6.3f} {rec:6.3f} {f1:6.3f} {supp:8,}")
```

Tag	Prec	Rec	F1	Support
NOUN	0.943	0.862	0.901	34,030
VERB	0.968	0.924	0.946	32,291
ADJ	0.827	0.850	0.838	9,855
ADV	0.883	0.863	0.873	10,299
PRON	0.891	0.959	0.924	13,784
DET	0.926	0.975	0.950	20,030
ADP	0.897	0.960	0.927	18,434
NUM	0.938	0.918	0.928	1,072
CONJ	0.974	0.995	0.984	5,974
PRT	0.892	0.828	0.859	6,513
.	0.955	1.000	0.977	29,083
X	0.447	0.188	0.265	181

3c. Confusion Matrix

```
In [85]: conf = pd.DataFrame(0, index=TAGS, columns=TAGS)
        for g, p in zip(gold_tags_all, pred_tags_all):
            conf.loc[g, p] += 1

        print("Confusion matrix (rows = gold, cols = predicted):")
        print(conf.to_string())

        # Top off-diagonal confusions
        errors = [
            (conf.loc[g, p], g, p)
            for g in TAGS for p in TAGS
            if g != p and conf.loc[g, p] > 0
        ]
        errors.sort(reverse=True)

        print("\nTop 10 confusions (gold → predicted, count):")
        for count, g, p in errors[:10]:
            print(f"    {g} → {p}: {count:,}")
```

Confusion matrix (rows = gold, cols = predicted):

	NOUN	VERB	ADJ	ADV	PRON	DET	ADP	NUM	CONJ	PRT	.	X
NOUN	29339	610	817	167	1209	653	235	63	43	79	791	24
VERB	956	29845	447	120	47	261	319	0	33	4	256	3
ADJ	427	198	8381	438	29	180	56	0	6	6	128	6
ADV	177	107	417	8893	24	49	261	0	28	255	86	2
PRON	15	4	4	0	13221	334	198	0	0	3	5	0
DET	1	4	1	20	250	19529	212	2	7	0	2	2
ADP	12	19	22	326	18	12	17690	0	25	306	2	2
NUM	50	9	8	2	1	7	5	984	0	2	4	0
CONJ	1	1	0	3	0	18	3	0	5946	0	2	0
PRT	76	28	28	103	32	25	734	0	18	5395	71	3
.	0	0	0	0	0	0	0	0	0	0	29083	0
X	64	6	14	4	8	14	5	0	1	1	30	34

Top 10 confusions (gold → predicted, count):

NOUN → PRON: 1,209

VERB → NOUN: 956

NOUN → ADJ: 817

NOUN → .: 791

PRT → ADP: 734

NOUN → DET: 653

NOUN → VERB: 610

VERB → ADJ: 447

ADJ → ADV: 438

ADJ → NOUN: 427

```
In [86]: test_pred = pd.DataFrame({
    "words": word_tags_all,
    "gold": gold_tags_all,
    "pred": pred_tags_all
})
frequent_errors = test_pred[(test_pred["gold"] == "NOUN") & (test_pred["pred"] == "
frequent_errors[(frequent_errors > 5) & (frequent_errors < 15)]
```

```

Out[86]: words
eugenia      13
dolores      13
russ         13
theresa      12
maggie       12
pamela       12
waddell      12
cobb         11
ernie        10
keith        9
lucy         9
ryan         9
feathertop   9
bobbie       9
eddie        9
benson       9
frankie      8
pete         8
pat          8
nicolas      8
gavin        8
freddy       8
hal          8
b'dikkat     8
nadine       8
welch        7
johnnie      7
chris        7
susan        7
tommy        7
spencer      7
myra         7
dan          7
montero      7
clayton      7
hague       7
rod          7
jubal        6
cathy        6
hesperus     6
cappy        6
doc          6
tilghman     6
delphine     6
artie        6
penny        6
todman       6
joyce        6
Name: count, dtype: int64

```

Question 3.1. What is your overall token accuracy? A `DefaultTagger` (always predicting NOUN) achieves ~13%; a state-of-the-art neural tagger exceeds 97%. Where does your HMM fall, and why?

Question 3.2. Which tag has the lowest F1 score? Examine 10 examples of misclassified tokens for that tag. Is the error systematic (always confused with the same other tag) or scattered?

Question 3.3. Identify the single most common confusion (largest off-diagonal cell). Give three English words that frequently trigger this confusion and explain why the HMM gets them wrong.

Write your answers here:

3.1. The held out token accuracy (an estimate for the token accuracy) is 92.7% which is better than the DefaultTagger (which makes sense because it can tag words as POS other than NOUNs) and less than the SOTA. This makes sense because sequential neural networks can capture more complex relationships between words in a sequence providing more accurate transition accuracies.

3.2. The lowest F1 score is the X tag. This makes sense because tokens classified as X are edge cases and make up a small fraction of the dataset. Looking at the confusion matrix X tagged words are commonly confused for punctuation with a bit of scatter. This is because many of the X words are foreign and contain punctuation: ['chelas', 'matsyendra', 'l'unita', 'hoy', 'l'osservatore', 'romano', 'ksu'u'peli'afo', 'mai'teipa', 'et', 'ceteras']. If we instead look at the "real tags", ADJ has the lowest F1 score, which seems to frequently be confused with nouns via the confusion matrix, not much scatter. Examples: ['demoniac', 'slick', 'curt', 'untidy', 'fine-boned', 'morose', 'deceased', 'further', 'illicit', 'buccolic']

3.3. Looking at the confusion matrix (which isn't normalize so this is biased towards frequent gold tags) is NOUN-PRON. This almost entirely seems to be due to names mistakenly categorized as pronouns like "Curt", "Phil", and "Matsuo".

Part 4 — OOV Handling with Suffix Heuristics

Words not seen during training receive only a tiny uniform floor from `emit_p`. Suffix patterns are a strong signal for unseen words.

4a. OOV Rate on Test Data

```
In [87]: oov_tokens = [
          (w.lower(), t)
          for sent in test_sents
          for w, t in sent
          if w.lower() not in train_vocab
        ]
oov_total = sum(len(s) for s in test_sents)

print(f"OOV tokens in Brown test set: {len(oov_tokens):,} / {oov_total:,} ")
```

```

f"= {len(oov_tokens)/oov_total:.3f}")

print("\nGold tag distribution for OOV words:")
oov_tag_dist = Counter(t for _, t in oov_tokens)
for tag, count in oov_tag_dist.most_common():
    print(f" {tag:6s} {count:5,} ({count/len(oov_tokens):.3f})")

```

OOV tokens in Brown test set: 6,438 / 181,546 = 0.035

Gold tag distribution for OOV words:

NOUN	4,298	(0.668)
VERB	908	(0.141)
ADJ	725	(0.113)
ADV	210	(0.033)
PRT	129	(0.020)
X	119	(0.018)
ADP	16	(0.002)
NUM	14	(0.002)
CONJ	8	(0.001)
DET	7	(0.001)
PRON	4	(0.001)

4b. Suffix Rule Table

Suffix	Most likely tag	Example
-ing	VERB	<i>running, increasing</i>
-ed	VERB	<i>walked, improved</i>
-ly	ADV	<i>quickly, strongly</i>
-tion , -sion	NOUN	<i>nation, decision</i>
-ity , -ty	NOUN	<i>quality, beauty</i>
-ous , -ious	ADJ	<i>famous, previous</i>
-able , -ible	ADJ	<i>available, possible</i>
-er , -or	NOUN	<i>teacher, actor</i>
All digits	NUM	<i>42, 1984</i>

4c. Suffix-Aware Emission Function

```

In [88]: def suffix_tag_probs(word):
    """
    Returns dict {tag: weight} for OOV words based on suffix/shape.
    Returns None if no rule fires.
    """
    w = word.lower()
    if w.isdigit():
        return {'NUM': 0.95, 'NOUN': 0.04, 'X': 0.01}
    if w.endswith(('tion', 'sion', 'ity', 'ty', 'ment', 'ness', 'er', 'or')):
        return {'NOUN': 0.75, 'VERB': 0.10, 'ADJ': 0.10, 'X': 0.05}

```

```

    if w.endswith('ing'):
        return {'VERB': 0.60, 'NOUN': 0.25, 'ADJ': 0.10, 'X': 0.05}
    if w.endswith('ed'):
        return {'VERB': 0.65, 'ADJ': 0.25, 'NOUN': 0.05, 'X': 0.05}
    if w.endswith('ly'):
        return {'ADV': 0.80, 'ADJ': 0.10, 'NOUN': 0.05, 'X': 0.05}
    if w.endswith(('ous', 'ious', 'able', 'ible', 'al', 'ful')):
        return {'ADJ': 0.80, 'NOUN': 0.10, 'ADV': 0.05, 'X': 0.05}
    return None

OOV_FLOOR = 1e-6

def emit_p_oov(tag, word):
    """Emission probability using suffix heuristics for OOV words."""
    w = word.lower()
    if w in train_vocab:
        return emit_p(tag, word)
    priors = suffix_tag_probs(word)
    if priors is None:
        return OOV_FLOOR
    return priors.get(tag, OOV_FLOOR)

```

4d. Re-Evaluate with OOV Heuristics

```

In [89]: def tag_sentence_oov(words):
    return viterbi([w.lower() for w in words], TAGS, start_p, trans_p, emit_p)

correct_oov = 0
total_oov = 0

for sent in test_sents:
    words = [w.lower() for w, t in sent]
    gold = [t for w, t in sent]
    pred = tag_sentence_oov(words)
    for w, g, p in zip(words, gold, pred):
        if w not in train_vocab:
            total_oov += 1
            if g == p:
                correct_oov += 1

print(f"OOV accuracy before suffix heuristics: "
      f"{correct_oov}/{total_oov} = {correct_oov/total_oov:.4f}")

```

OOV accuracy before suffix heuristics: 2,375/6,438 = 0.3689

```

In [90]: def tag_sentence_oov(words):
    return viterbi([w.lower() for w in words], TAGS, start_p, trans_p, emit_p_oov)

correct_oov = 0
total_oov = 0

for sent in test_sents:
    words = [w.lower() for w, t in sent]
    gold = [t for w, t in sent]
    pred = tag_sentence_oov(words)

```

```

for w, g, p in zip(words, gold, pred):
    if w not in train_vocab:
        total_oov += 1
        if g == p:
            correct_oov += 1

print(f"OOV accuracy after suffix heuristics: "
      f"{correct_oov:}/{total_oov:} = {correct_oov/total_oov:.4f}")

```

OOV accuracy after suffix heuristics: 3,634/6,438 = 0.5645

```

In [91]: all_rules = [
    'isdigit',
    'tion', 'sion', 'ity', 'ty', 'ment', 'ness', 'er', 'or', # Nouns
    'ing', 'ed', 'ly', # Verbs/Adverbs
    'ous', 'ious', 'able', 'ible', 'al', 'ful', # Adjectives
    'No_Rule_Fired'
]

suffix_stats = {rule: {'correct': 0, 'total': 0} for rule in all_rules}

def get_matched_rule(word):
    """Checks word against individual suffixes in the exact order of the original c
    w = word.lower()

    if w.isdigit():
        return 'isdigit'

    for suff in ('tion', 'sion', 'ity', 'ty', 'ment', 'ness', 'er', 'or'):
        if w.endswith(suff):
            return suff

    if w.endswith('ing'):
        return 'ing'

    if w.endswith('ed'):
        return 'ed'

    if w.endswith('ly'):
        return 'ly'

    for suff in ('ous', 'ious', 'able', 'ible', 'al', 'ful'):
        if w.endswith(suff):
            return suff

    return 'No_Rule_Fired'

correct_oov = 0
total_oov = 0

for sent in test_sents:
    words = [w.lower() for w, t in sent]
    gold = [t for w, t in sent]
    pred = tag_sentence_oov(words)

    for w, g, p in zip(words, gold, pred):

```

```

if w not in train_vocab:
    total_oov += 1
    if g == p:
        correct_oov += 1

    rule = get_matched_rule(w)
    suffix_stats[rule]['total'] += 1
    if g == p:
        suffix_stats[rule]['correct'] += 1

print(f"Overall OOV accuracy: {correct_oov:,}/{total_oov:,} = {correct_oov/total_oov:.4f}")

print(f"{'Suffix/Rule':<15} | {'Correct':<8} | {'Total':<8} | {'Accuracy'}")
print("-" * 48)

sorted_stats = sorted(suffix_stats.items(), key=lambda item: item[1]['total'], reverse=True)

for rule, stats in sorted_stats:
    if stats['total'] > 10:
        acc = stats['correct'] / stats['total']
        print(f"{'rule':<15} | {stats['correct']:<8} | {stats['total']:<8} | {acc:.4f}")

```

Overall OOV accuracy: 3,634/6,438 = 0.5645

Suffix/Rule	Correct	Total	Accuracy
No_Rule_Fired	2360	4773	0.4944
ed	490	626	0.7827
ing	215	363	0.5923
er	202	238	0.8487
ly	165	194	0.8505
al	35	43	0.8140
tion	31	32	0.9688
ness	31	31	1.0000
or	27	31	0.8710
ous	20	26	0.7692
ty	12	25	0.4800
ity	12	14	0.8571
ful	9	14	0.6429

Question 4.1. What fraction of Brown test-set OOV words are NOUNs? Does the suffix heuristic table reflect this distribution, or does it over-weight any tag?

Question 4.2. How much does adding suffix heuristics improve OOV accuracy? Identify two suffixes that help the most and one that is misleading.

Question 4.3. The capitalization heuristic (capitalized non-sentence-initial word → NOUN) works well for newswire but perhaps breaks on Twitter/Bluesky. Give two categories of capitalized words common in social media that are NOT nouns. Can you produce evidence from something in the data or your analysis that supports the idea that the heuristic “breaks”; or can you say that it still has value, based on your analysis or your own study of these data?

Write your answers here:

4.1. 66.8% of the OOV words are NOUNS. The suffix rule table over-weights adjectives which have 6 suffixes that indicate a high probability of adjectives even though they are ~11% of the data. Nouns have 8.

4.2. Using the standard emit_p function on OOV we get an accuracy of 0.3689, with emit_p_oov we get an accuracy of 0.5645. Looking at each suffix that occurs at least 10 times in the OOV set, words that had a -ness suffix had 100% accuracy after the heuristic. -tion was the second best with 96.88%. -ty is the most misleading this is probably because a lot of adjectives end in -ty like rusty or crusty.

4.3. One category that would break that capitalization heuristic is causal or expressive typing. For example someone may say "I'm SO done with election season" This doesn't mean SO is a name and breaks the heuristic. This also works for acronyms which are typically capitalized like TBH or OMG. We can also find examples in the above dataset that this doesn't work, for example "Midwestern" is a adjective that is typically capitalized outside of the start of a sentence.

```
In [92]: gold_tags_all = []
word_tags_all = []

for sent in test_sents:
    if not sent:
        continue
    words = [w for w, t in sent]
    gold = [t for w, t in sent]

    gold_tags_all.extend(gold)
    word_tags_all.extend(words)
```

```
In [93]: df = pd.DataFrame({
    "words": word_tags_all,
    "gold": gold_tags_all
})
df[df["words"].str[0].str.isupper()]
```

Out[93]:

	words	gold
0	The	DET
9	I	PRON
16	Oh	PRT
20	I'm	PRT
29	Beech	NOUN
...	...	...
181469	She	PRON
181491	Cherokee	NOUN
181495	Midwestern	ADJ
181522	From	ADP
181524	I	PRON

19273 rows × 2 columns

Part 5 — Tagging Bluesky Skeets

5a. Load and Tokenize Skeets

```
In [94]: from google.cloud import bigquery

client = bigquery.Client(project="stanford-f24-datasci-194d")

QUERY = """
SELECT text
FROM `stanford-f24-datasci-194d.EMS.bsky-firehose`
WHERE JSON_VALUE(post_json, '$.record.langs[0]') = 'en'
      AND DATE(timestamp) BETWEEN '2024-11-18' AND '2024-11-24'
      AND MOD(ABS(FARM_FINGERPRINT(CAST(sequence AS STRING))), 100) < 1
LIMIT 5000
"""

df_skeet = client.query(QUERY).to_dataframe()
print(f"Loaded {len(df_skeet):,} skeets")

def tokenize_skeet(text):
    """Light tokenization: strip URLs, keep hashtags and @-mentions as tokens."""
    text = re.sub(r'https?://\S+', ' ', text)
    return re.findall(r'#\w+|@\w+|[a-zA-Z]+\d+|[\s\w]', text)

# Preview
for text in df_skeet['text'].dropna().head(3):
```

```
print(f"Text: {text[:100]}")
print(f"Tokens: {tokenize_skeet(text)}")
print()
```

c:\Users\ricky\miniforge3\envs\stats292\Lib\site-packages\google\auth_default.py:114: UserWarning: Your application has authenticated using end user credentials from Google Cloud SDK without a quota project. You might receive a "quota exceeded" or "API not enabled" error. See the following page for troubleshooting: <https://cloud.google.com/docs/authentication/adc-troubleshooting/user-creds>.

```
warnings.warn(_CLOUD_SDK_CREDENTIALS_WARNING)
Loaded 5,000 skeets
Text:
Tokens: []
```

```
Text: you're welcome 🤗
Tokens: ['you', '', 're', 'welcome', '🤗']
```

```
Text: Sure thing, Boss.
Tokens: ['Sure', 'thing', ',', 'Boss', '.']
```

5b. OOV Rate on Bluesky

```
In [95]: skeet_tokens = []
for text in df_skeet['text'].dropna():
    skeet_tokens.extend(tokenize_skeet(text))

skeet_vocab = set(t.lower() for t in skeet_tokens if t.isalpha())
oov_in_skeet = skeet_vocab - train_vocab

print(f"Unique word types in skeet sample: {len(skeet_vocab):,}")
print(f"Types not in Brown training vocab: {len(oov_in_skeet):,} "
      f"({len(oov_in_skeet)/len(skeet_vocab):.3f})")
print(f"\nSample OOV words:")
for w in sorted(oov_in_skeet)[:30]:
    print(f" {w}")
```

Unique word types in skeet sample: 11,996
Types not in Brown training vocab: 4,716 (0.393)

Sample OOV words:

aaaa
aaaaa
aaaaaaa
aaages
aall
abandonn
abbotsbury
abcrn
abd
aberdeen
ableism
ableist
ablut
abominable
absofuckinlutley
absurdo
abt
abuelita
abusers
ac
aca
accelerationist
accessing
acct
accts
aceito
achava
acho
aclu
acm

5c. Tag a Sample of Skeets

```
In [96]: sample_texts = df_skeet['text'].dropna().sample(200, random_state=42).tolist()
tagged_skeets = []

for text in sample_texts:
    tokens = tokenize_skeet(text)
    if not tokens:
        continue
    words = [t.lower() for t in tokens]
    try:
        tags = tag_sentence_oov(words)
    except Exception:
        tags = ['X'] * len(words)
    tagged_skeets.append(list(zip(tokens, tags)))

print("First 10 tagged skeets:")
for tagged in tagged_skeets[:10]:
    print(" ".join(f"{w}/{t}" for w, t in tagged))
print()
```

First 10 tagged skeets:

Hate/NOUN to/ADP sound/ADJ snarky/NOUN ,/. but/CONJ .../DET .../NOUN

"/DET Guys/NOUN ,/. it/PRON is/VERB alright/NOUN ./.. The/DET child/NOUN was/VERB actually/ADV a/DET goblin/NOUN or/CONJ something/NOUN ./.. "/NOUN -/ADP Destiny/NOUN in/ADP the/DET near/ADJ future/NOUN

music/NOUN ./.. apple/NOUN ./.. com/VERB //ADP us/PRON //ADP album/NOUN //ADP my/DET -/NOUN ./.. ./.. ./..

We/PRON sure/ADV have/VERB quite/ADV healthy/ADJ habits/NOUN I/PRON might/VERB say/VERB !/..

Shoutout/NOUN to/ADP @martin/NOUN ./.. kleppmann/NOUN ./.. com/. !/..

reply/NOUN guy/NOUN ?/.. DECRY/DET GUY/NOUN

👉/DET

Killing/DET Eve/NOUN ./.. ./.. ./.. did/VERB not/ADV nail/ADP the/DET landing/NOUN ./.. If/ADP you/PRON haven/VERB '/. t/NOUN spoiled/VERB yourself/PRON about/ADP the/DET end/NOUN ,/. I/PRON would/VERB let/VERB the/DET first/ADJ few/ADJ seasons/NOUN be/VERB a/DET happy/ADJ memory/NOUN and/CONJ leave/VERB it/PRON at/ADP that/DET 🙄/NOUN

I/PRON just/ADV don/NOUN '/. t/NOUN have/VERB the/DET hankering/NOUN ./.. The/DET line/NOUN is/VERB nuts/DET long/ADJ over/ADP there/PRT

Because/ADP they/PRON are/VERB guilty/ADJ of/ADP war/NOUN crimes/NOUN themselves/PRON N ./..

Hate/NOUN to/ADP sound/ADJ snarky/NOUN ,/. but/CONJ .../DET .../NOUN

"/DET Guys/NOUN ,/. it/PRON is/VERB alright/NOUN ./.. The/DET child/NOUN was/VERB actually/ADV a/DET goblin/NOUN or/CONJ something/NOUN ./.. "/NOUN -/ADP Destiny/NOUN in/ADP the/DET near/ADJ future/NOUN

music/NOUN ./.. apple/NOUN ./.. com/VERB //ADP us/PRON //ADP album/NOUN //ADP my/DET -/NOUN ./.. ./.. ./..

We/PRON sure/ADV have/VERB quite/ADV healthy/ADJ habits/NOUN I/PRON might/VERB say/VERB !/..

Shoutout/NOUN to/ADP @martin/NOUN ./.. kleppmann/NOUN ./.. com/. !/..

reply/NOUN guy/NOUN ?/.. DECRY/DET GUY/NOUN

👉/DET

Killing/DET Eve/NOUN ./.. ./.. ./.. did/VERB not/ADV nail/ADP the/DET landing/NOUN ./.. If/ADP you/PRON haven/VERB '/. t/NOUN spoiled/VERB yourself/PRON about/ADP the/DET end/NOUN ,/. I/PRON would/VERB let/VERB the/DET first/ADJ few/ADJ seasons/NOUN be/VERB a/DET happy/ADJ memory/NOUN and/CONJ leave/VERB it/PRON at/ADP that/DET 🙄/NOUN

I/PRON just/ADV don/NOUN '/. t/NOUN have/VERB the/DET hankering/NOUN ./.. The/DET line/NOUN is/VERB nuts/DET long/ADJ over/ADP there/PRT

Because/ADP they/PRON are/VERB guilty/ADJ of/ADP war/NOUN crimes/NOUN themselves/PRO
N ./.

5d. Failure Mode Analysis

```
In [97]: hashtags = []
mentions = []
tagged_x = []

for tagged in tagged_skeets:
    for word, tag in tagged:
        if word.startswith('#'):
            hashtags.append((word, tag))
        elif word.startswith('@'):
            mentions.append((word, tag))
        elif tag == 'X':
            tagged_x.append(word)

print(f"Hashtags: {len(hashtags):,}")
print(f" Tag distribution: {Counter(t for _, t in hashtags).most_common()}")
print(f"\n@-Mentions: {len(mentions):,}")
print(f" Tag distribution: {Counter(t for _, t in mentions).most_common()}")
print(f"\nTokens tagged X: {len(tagged_x):,}")
print(f" Sample: {tagged_x[:20]}")
```

Hashtags: 13
Tag distribution: [('NOUN', 5), ('DET', 4), ('.', 2), ('ADV', 1), ('VERB', 1)]

```
@-Mentions: 6
Tag distribution: [('NOUN', 6)]
```

Tokens tagged X: 33

Sample: ['u', 'q', 'q', 'l', 'l', 'q', 'q', 'y', 'q', 'p', 'l', 'l', 'l', 'p', 'l', 'p', 'q', 'q', 'l', 'l', 'l', 'y', 'l']

5e. Tag Distribution: Brown vs. Bluesky

```
In [98]: brown_dist = Counter(gold_tags_all)
brown_total = sum(brown_dist.values())

skeet_all_tags = [tag for sent in tagged_skeets for _, tag in sent]
skeet_dist = Counter(skeet_all_tags)
skeet_total = sum(skeet_dist.values())

print(f"{'Tag':6s} {'Brown %':>8} {'Bluesky %':>10} {'Ratio':>6}")
print("-" * 40)
for tag in TAGS:
    b = brown_dist[tag] / brown_total
    s = skeet_dist[tag] / skeet_total if skeet_total > 0 else 0
    ratio = s / b if b > 0 else float('inf')
    print(f"{'tag':6s} {'b':8.3f} {'s':10.3f} {'ratio':6.2f}")
```

Tag	Brown %	Bluesky %	Ratio
NOUN	0.187	0.213	1.14
VERB	0.178	0.152	0.86
ADJ	0.054	0.061	1.12
ADV	0.057	0.055	0.98
PRON	0.076	0.081	1.07
DET	0.110	0.107	0.97
ADP	0.102	0.112	1.10
NUM	0.006	0.016	2.73
CONJ	0.033	0.022	0.67
PRT	0.036	0.023	0.65
.	0.160	0.149	0.93
X	0.001	0.008	8.34

Question 5.1. What is the OOV rate for your Bluesky sample? How does it compare to the Brown test-set OOV rate?

Question 5.2. What tag does your tagger most often assign to hashtags (e.g., #blessed)? Is this linguistically defensible? What tag do @-mentions receive?

Question 5.3. Compare NOUN and VERB proportions in Brown (gold) vs. Bluesky (predicted). Does the tag distribution reflect that Bluesky text is more conversational, and in which direction?

Question 5.4. Find 5 skeet tokens your tagger clearly mislabels. For each, state the predicted tag, the likely correct tag, and the source of error (OOV, ambiguity, domain shift, or tokenization).

Write your answers here:

5.1. There is a 40% OOV rate which is much higher than the Brown test-set OOV (<4%)

5.2. hashtags are commonly assigned noun but also det or punctation. Noun makes sense (I am feeling #blessed), but DET and PUNCT aren't defensible. @-mentions are all nouns which makes sense.

5.3. Brown has a lower proportion of Nouns and higher proportion of Verbs than Bluesky this doesn't indicate that Bluesky is more conversational as we'd expect coversational corpi to contain more verbs than non-conversational ones.

5.4. "Hate to sound snarky" is completely mislabeled, with hate being labeled a noun when it is a verb. This is due to ambiguity due to the pronoun drop (I) Hate ... "👉" is misclassified as DET which is due to OOV it should be either X or PUNCT. "Killing Eve" was labeled Det Noun when it should have been Noun Noun as the name of a TV show, this due to domain shift as Killing probably isn't used like that before. In "the line is nuts long" nuts is misclassified as DET when it is an adverb modifying long, this is due to doman shift. Finally the tokenizer breaks up URLs like music.apple.com/us/album/ and then the model has no idea how to treat them each part. Ideally the tokenizer should keep the whole thing together and just

label it as a NOUN since the individual words aren't a part of the sentence.

Part 6 — Error Analysis and Discussion

Question 6.1. The Brown Corpus is American English, edited, from 1964. Name three word types or constructions that appear in Bluesky that did not exist in 1964, and explain how each stresses the HMM.

Question 6.2. The Viterbi algorithm finds the globally optimal tag sequence. The greedy approach (always pick the locally best tag) is simpler. Under what circumstances would greedy tagging produce the same result as Viterbi? When would they diverge most?

Question 6.3. The HMM transition matrix encodes only first-order dependencies (each tag depends only on the previous tag). What is the number of parameters in the transition matrix for a 12-tag first-order model? For a second-order model? What are the statistical consequences of moving to second order on a corpus the size of Brown?

Question 6.4. Modern neural POS taggers (e.g., spaCy's transformer pipeline) exceed 98% accuracy on English newswire. Identify two structural limitations of the HMM that prevent it from reaching 98%, and describe how a contextual embedding model addresses each.

Write your answers here:

6.1. URLs are a new construction that stress the HMM because they are highly unique and so will likely be OOV. Emojis are a new word type that would stress the HMM with OOV if not trained on emojis but even if they are emoji's add a new dimension that aren't as clear cut as they can sometimes act as punctuation and sometimes they convey meaning of their own acting as a verb, adj, noun, etc. Finally hastags stress the HMM because normal tokenization rules that split the hashtag from the rest of the word don't work but then #blessed and blessed could play very different roles in a sentence stressing the HMM.

6.2. They would produce the same result when there is little ambiguity or the emission probability is large that both algorithms output the same. They'll diverge in very ambiguous settings like those that contain words with many different high probability meanings Viterbi is able to realize that some of those tags won't match the rest of the sentence but the greedy algo can't.

6.3. A 12-tag first order model has 144 parameters. A second-order model has 1728 parameters. Higher order models have much sparser transition matrices so many transitions will have the OOV lower bound probability. Since Brown isn't that large it would likely have more sparsity/

6.4. HMMs cannot resolve long distance dependencies because it only models local transition probabilities. A contextual embedding model can add information from long distance

dependencies into the current word. Another issue with HHMs is that they don't have any semantic understanding which makes it harder for them to deal with OOV words. With a contextual embedding model you can tokenize more aggressively and find in-vocabulary roots/prefixes/suffixes that provide information about the word.

Part 7 — Extension: Real-Word Error Detection (Optional)

A real-word error is a correctly spelled token that is the wrong word (e.g., *their* vs. *there*). POS context can flag positions where the tagger is unusually uncertain — a low margin between the top-1 and top-2 tag scores at an in-vocabulary word signals possible real-word confusion.

```
In [99]: def flag_real_word_errors(words, threshold=0.9):
        """
        Run Viterbi and flag positions where the log-prob margin between
        the best and second-best tag is below `threshold`.
        Only flags in-vocabulary words (OOV words are always uncertain).

        Returns list of (position, word, predicted_tag, margin).
        """
        n = len(words)
        viterbi_scores = [{}]
        backptr = [{}]

        for tag in TAGS:
            viterbi_scores[0][tag] = (math.log(start_p[tag]) +
                                     math.log(emit_p_oov(tag, words[0])))
            backptr[0][tag] = None

        for t in range(1, n):
            viterbi_scores.append({})
            backptr.append({})
            word = words[t]
            for curr_tag in TAGS:
                log_emit = math.log(emit_p_oov(curr_tag, word))
                best_prev, best_score = None, float('-inf')
                for prev_tag in TAGS:
                    score = (viterbi_scores[t-1][prev_tag] +
                           math.log(trans_p[prev_tag][curr_tag]) +
                           log_emit)
                    if score > best_score:
                        best_score = score
                        best_prev = prev_tag
                viterbi_scores[t][curr_tag] = best_score
                backptr[t][curr_tag] = best_prev

        suspects = []
        for t in range(n):
            sorted_scores = sorted(viterbi_scores[t].values(), reverse=True)
```

```

        margin = sorted_scores[0] - sorted_scores[1] if len(sorted_scores) > 1 else
        best_tag = max(viterbi_scores[t], key=viterbi_scores[t].get)
        if margin < threshold and words[t].lower() in train_vocab:
            suspects.append((t, words[t], best_tag, round(margin, 3)))
    return suspects

```

Example

```

example = "there going to the store to buy food".split()
predicted = tag_sentence_oov(example)
suspects = flag_real_word_errors(example)

print("Sentence: ", " ".join(example))
print("Tags:      ", " ".join(predicted))
print("Suspects: ", suspects)

```

Sentence: there going to the store to buy food

Tags: PRT VERB ADP DET NOUN PRT VERB NOUN

Suspects: [(0, 'there', 'PRT', 0.876), (2, 'to', 'PRT', 0.144)]

In [100...

```

def flag_real_word_errors(words, threshold=2):
    """
    Run Viterbi and flag positions where the log-prob margin between
    the best and second-best tag is below `threshold`.
    Only flags in-vocabulary words (OOV words are always uncertain).

    Returns list of (position, word, predicted_tag, margin).
    """
    n = len(words)
    viterbi_scores = [{}]
    backptr = [{}]

    for tag in TAGS:
        viterbi_scores[0][tag] = (math.log(start_p[tag]) +
                                   math.log(emit_p_oov(tag, words[0])))
        backptr[0][tag] = None

    for t in range(1, n):
        viterbi_scores.append({})
        backptr.append({})
        word = words[t]
        for curr_tag in TAGS:
            log_emit = math.log(emit_p_oov(curr_tag, word))
            best_prev, best_score = None, float('-inf')
            for prev_tag in TAGS:
                score = (viterbi_scores[t-1][prev_tag] +
                        math.log(trans_p[prev_tag][curr_tag]) +
                        log_emit)
                if score > best_score:
                    best_score = score
                    best_prev = prev_tag
            viterbi_scores[t][curr_tag] = best_score
            backptr[t][curr_tag] = best_prev

    suspects = []
    for t in range(n):
        sorted_scores = sorted(viterbi_scores[t].values(), reverse=True)

```

```

margin = sorted_scores[0] - sorted_scores[1] if len(sorted_scores) > 1 else
best_tag = max(viterbi_scores[t], key=viterbi_scores[t].get)
if margin < threshold and words[t].lower() in train_vocab:
    suspects.append((t, words[t], best_tag, round(margin, 3)))
return suspects

# Example
example = "they're going 2 tha store ta buy food".split()
predicted = tag_sentence_oov(example)
suspects = flag_real_word_errors(example)

print("Sentence: ", " ".join(example))
print("Tags:      ", " ".join(predicted))
print("Suspects: ", suspects)

```

```

Sentence:  they're going 2 tha store ta buy food
Tags:      PRT VERB NUM NOUN NOUN . VERB NOUN
Suspects:  [(4, 'store', 'NOUN', 1.408)]

```

Question 7.1. Does your detector flag `there` in the example above? What margin score does it receive? What would need to be true of the transition probabilities for the detector to reliably catch this error?

Question 7.2. How do your results on 7.1 change if you change the test phrase to `"they're going 2 tha store ta buy food"` ?

Write your answers here:

7.1. No, it doesn't suspect `there` . It receives a margin score of 0.876. For the detector to reliably catch this error, the typo "there" would have to cause ambiguity between the top-1 and top-2 tag scores which mean the transition probabilities would have to be close/competitive with each other.

7.2. The margins for the words become much larger. This is likely because the transition probabilities become very small for OOV words like `tha` or `ta` in unusual context (using 2), so there isn't much of a margin between labeling options.

Deliverables

Submit this notebook with all cells executed:

1. The 12×12 transition matrix A printed as a table.
2. Overall token accuracy on held-out Brown, plus per-tag precision/recall/F1.
3. The confusion matrix, with the top 5 off-diagonal entries identified and explained.
4. OOV accuracy before and after suffix heuristics.
5. Viterbi output on at least 5 Bluesky skeets, with the OOV rate and tag distribution comparison table.
6. Written answers to all numbered questions (2–4 sentences each).

Ethics & Bias Reflection

Question E1. The Brown Corpus is American English, edited, compiled from the 1960's. The POS tagger you have constructed is trained using this corpus. Presumably this tagger is biased in the following manner. If used to tag Bluesky skeets, it may produce tag frequencies that resemble the frequencies the same tagger would produce on the Brown Corpus. Presumably any non-resemblance is 'real' and not bias, while any resemblance might be bias. How might you check whether some specific resemblance — say the tagging frequency of NOUN — is not bias?

Question E2. Modern neural POS taggers use drastically more compute and are trained on drastically more data than what we just did. What can you say (for example by doing outside reading) about the resource cost (compute, storage, human labelling) involved in constructing one of those taggers? What can you say about the value to society of spending such levels of resources? If the level of resources required to go from 80% accuracy to 98% is 10x, 100x, 1000x, 10000x, etc. At what point does it become 'not ethical' to pursue the project?

Write your answers here:

E1. The one way to check that the resemblance is not bias is to create your own groundtruth using a subsample of the Bsky data. Then you can calculate the tagger's accuracy on the Bsky data and see if it's just resemblance or actually bias. You could also analyze the OOV tagging the results for large scale issues (like labelling everything at as a NOUN at a much higher rate than expected.)

E2. As far as training a modern neural POS tagger like spaCy, the compute and storage demands are relatively modest using modern hardware, however this becomes more of a concern with LLMs. However the human labelling demands can be quite large and expensive (training, verification, etc) and may rely on exploitative labor practices (outsourcing to developing nations, illegally/unethically webscraped data, etc) The value of the project really depends on the use and necessity of the taggers. If making the taggers more accurate make technology more accessible to "edge" users which may be historically disenfranchised (such as AAVE) then even a 100x-1000x increase in resources used may be worthwhile. Since errors are typically systemic and not random, even a 98% accuracy can be lead to users encountering frequent errors across millions of pieces of text.